

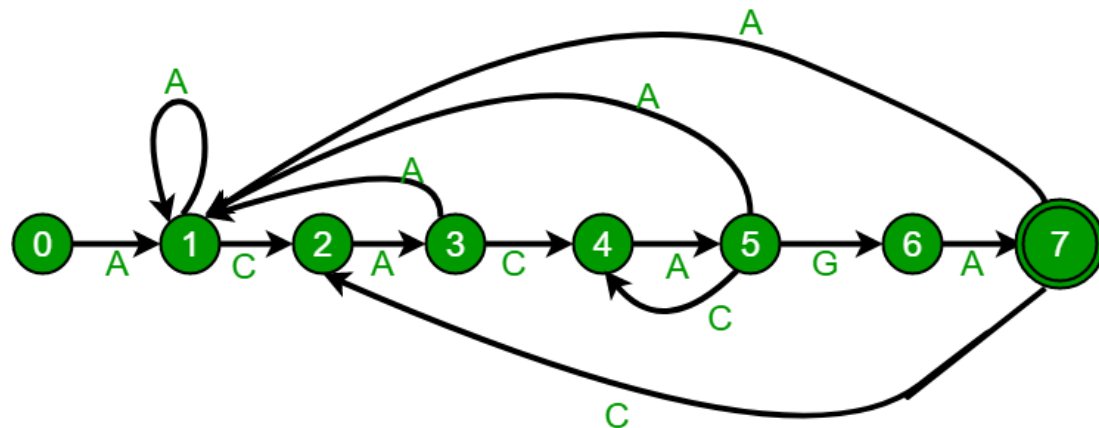
STRING MATCHING

String Matching with finite automata

Design and Analysis of Algorithm

FINITE-STATE AUTOMATON

- A **finite-state machine (FSM)** or **finite-state automaton (FSA**, plural: *automata*), **finite automaton**, or simply a **state machine**, is a mathematical model of computation.
- It is an abstract machine that can be in exactly one of a finite number of *states* at any given time.
- The FSM can change from one state to another in response to some inputs; the change from one state to another is called a *transition*



STRING MATCHING WITH FINITE AUTOMATA

- These algorithms build a finite automaton that scans the text string T for all occurrences of pattern P .
- Each text character is examined only once
- Time to build the automaton can be large if Σ is large.



FINITE AUTOMATA

- A *finite automaton* M is a 5-tuple $(Q, q_o, A, \Sigma, \delta)$, where
 - Q is finite set of *states*,
 - $q_o \in Q$ is the *start state*,
 - $A \subseteq Q$ is a distinguished set of *accepting states*,
 - Σ is finite *input alphabet*,
 - δ is a function from $Q \times \Sigma$ into Q , called the *transition function* of M



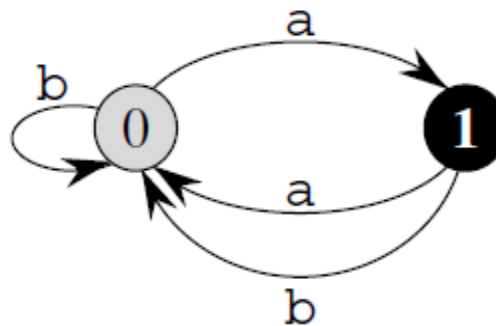
FINITE AUTOMATA (CONT'D)

- Suppose M is in state q_o .
- It reads char. a , it moves from state q to state $\delta(q,a)$
- Whenever current state $q \in A$, the machine M has *accepted* the string read so far.
- An input that is not accepted is said to be *rejected*



state	input	
	a	b
0	1	0
1	0	0

(a)



(b)

Figure 32.6 A simple two-state finite automaton with state set $Q = \{0, 1\}$, start state $q_0 = 0$, and input alphabet $\Sigma = \{a, b\}$. (a) A tabular representation of the transition function δ . (b) An equivalent state-transition diagram. State 1 is the only accepting state (shown blackened). Directed edges represent transitions. For example, the edge from state 1 to state 0 labeled b indicates $\delta(1, b) = 0$. This automaton accepts those strings that end in an odd number of a 's. More precisely, a string x is accepted if and only if $x = yz$, where $y = \varepsilon$ or y ends with a b , and $z = a^k$, where k is odd. For example, the sequence of states this automaton enters for input $abaaa$ (including the start state) is $\langle 0, 1, 0, 1, 0, 1 \rangle$, and so it accepts this input. For input $abbaa$, the sequence of states is $\langle 0, 1, 0, 0, 1, 0 \rangle$, and so it rejects this input.

FINAL-STATE FUNCTION

- The automaton M has a *final-state function* Φ from Σ^* to Q , such that:
- $\Phi(w)$ is the state, M ends up in, after scanning the string w .
- M accepts string w if and only if $\Phi(w) \in A$.
- It is defined recursively as follows:
 - $\Phi(w) = q_o$ if $w = \varepsilon$
 - $\Phi(wa) = \delta(\Phi(w), a)$ for $w \in \Sigma^*$, $a \in \Sigma$



STRING MATCHING AUTOMATA

- Every pattern P has finite automaton
- It must be built in the preprocessing step
- In order to do so, we first define a function called *suffix-function* σ , corresponding to P
- It is a mapping from Σ^* to $\{0, 1, \dots, m\}$ such that:
- $\sigma(x)$ = length of the longest prefix of P that is a suffix of x
- $\sigma(x) = \max\{k : P_k \sqsubseteq x\}$



STRING MATCHING AUTOMATA

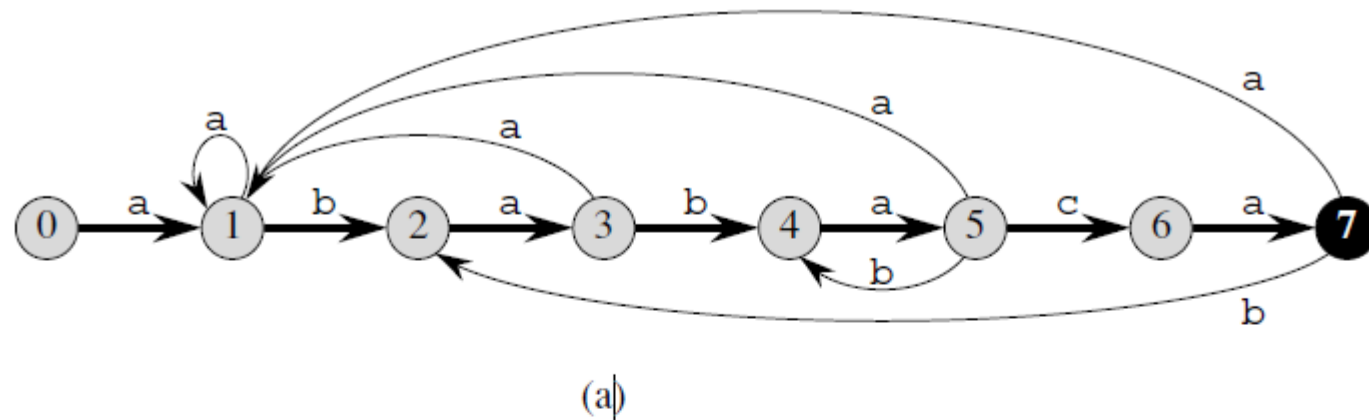
- Suffix function is well defined, since $P_0 = \varepsilon$ is a suffix of every string.
- If $P=ab$, then:
 - $\sigma(\varepsilon) = 0$
 - $\sigma(x) = 0$
 - $\sigma(ccaca) = 1, \sigma(ccab) = 2$



STRING MATCHING AUTOMATA

- $\sigma(x) = m$ if and only if P is a suffix of x
- String Matching Automaton corresponding to a given pattern is defined as:
 - State Set $Q = \{ 0, 1, 2, \dots, M \}$
 - Transition function
 - $\delta(q, a) = \sigma(P_q a)$





state	input			P
	a	b	c	
0	1	0	0	a
1	1	2	0	b
2	3	0	0	a
3	1	4	0	b
4	5	0	0	a
5	1	4	6	c
6	7	0	0	a
7	1	2	0	

$P = \bar{a}b\bar{a}b\bar{a}c\bar{a}$

i	—	1	2	3	4	5	6	7	8	9	10	11
$T[i]$	—	a	b	a	b	a	b	a	c	a	b	a
state $\phi(T_i)$	0	1	2	3	4	5	4	5	6	7	2	3

(b)

(c)

Fig 32.7

STRING MATCHING AUTOMATA

- Figure 32.7 (a) A state-transition diagram for the string-matching automaton that accepts all strings ending in the string ababaca. State 0 is the start state, and state 7 (shown blackened) is the only accepting state. A directed edge from state i to state j labeled a represents $\delta(i, a) = j$. The right-going edges forming the “spine” of the automaton, shown heavy in the figure, correspond to successful matches between pattern and input characters. The left-going edges correspond to failing matches. Some edges corresponding to failing matches are not shown; by convention, if a state i has no outgoing edge labeled a for some $a \in \Sigma$, then $\delta(i, a) = 0$. (b) The corresponding transition function δ , and the pattern string $P = \text{ababaca}$. The entries corresponding to successful matches between pattern and input characters are shown shaded. (c) The operation of the automaton on the text $T = \text{abababacaba}$. Under each text character $T[i]$ is given the state $\varphi(Ti)$ the automaton is in after processing the prefix Ti . One occurrence of the pattern is found, ending in position 9.



FINITE-AUTOMATON-MATCHER(T, δ, m)

```
1   $n \leftarrow \text{length}[T]$ 
2   $q \leftarrow 0$ 
3  for  $i \leftarrow 1$  to  $n$ 
4      do  $q \leftarrow \delta(q, T[i])$ 
5          if  $q = m$ 
6              then print “Pattern occurs with shift”  $i - m$ 
```



COMPUTE-TRANSITION-FUNCTION(P, Σ, m)

1 for $q = 0$ to m

2 for each character $a \in \Sigma$

3 $k = \min \{m, q + 1\}$

4 while $P[:k]$ is not a suffix of $P[:q]a$

5 $k = k - 1$

6 $\delta(q, a) = k$

7 return δ

COMPUTE-TRANSITION-FUNCTION(P, Σ, m)

```
1 for  $q = 0$  to  $m$ 
2   for each character  $a \in \Sigma$ 
3      $k = \min \{m, q + 1\}$ 
4     while  $P[:k]$  is not a suffix of  $P[:q]a$ 
5        $k = k - 1$ 
6    $\delta(q, a) = k$ 
7 return  $\delta$ 
```

The running time of **COMPUTE-TRANSITION-FUNCTION** is $O(m^3 |\Sigma|)$, because the outer loops contribute a factor of $m |\Sigma|$, the inner while loop can run at **most** $m + 1$ times, and the test for whether $P[:k]$ is a suffix of $P[:q]a$ on line 4 can require comparing up to m characters.

COMPUTE-TRANSITION-FUNCTION(P, Σ)

```
1   $m \leftarrow \text{length}[P]$ 
2  for  $q \leftarrow 0$  to  $m$ 
3      do for each character  $a \in \Sigma$ 
4          do  $k \leftarrow \min(m + 1, q + 2)$ 
5              repeat  $k \leftarrow k - 1$ 
6                  until  $P_k \sqsupseteq P_q a$ 
7                   $\delta(q, a) \leftarrow k$ 
8  return  $\delta$ 
```



COMPUTE-TRANSITION-FUNCTION(P, Σ)

```
1   $m \leftarrow \text{length}[P]$ 
2  for  $q \leftarrow 0$  to  $m$ 
3      do for each character  $a \in \Sigma$ 
4          do  $k \leftarrow \min(m + 1, q + 2)$ 
5              repeat  $k \leftarrow k - 1$ 
6                  until  $P_k \sqsupseteq P_q a$ 
7                   $\delta(q, a) \leftarrow k$ 
8  return  $\delta$ 
```

The running time of COMPUTE-TRANSITION-FUNCTION is $O(m^3 |\Sigma|)$,

the outer loops contribute a factor of $m |\Sigma|$.

the inner **repeat** loop can run at most $m+1$ times

and the test $P_k \sqsupseteq P_q a$ on line 6 can require comparing up to m characters



REFERENCE

○ Introduction to Algorithms

- Thomas H. Cormen
- Chapter # 32